

Title

Familiarizing with NumPy, Pandas, and Matplotlib

Aim

To learn the basics of NumPy, Pandas, and Matplotlib libraries in Python for numerical computation, data manipulation, and data visualization.

Aim

To learn the basics of NumPy, Pandas, and Matplotlib libraries in Python for numerical computation, data manipulation, and data visualization.

Objectives

- To create and manipulate 1-D and 2-D arrays using NumPy.
 - To perform indexing, slicing, and searching operations on arrays.
 - To calculate statistical measures such as mean, median, mode, variance, standard deviation, MSE, and RMSE.
 - To create and manipulate DataFrames using Pandas.
 - To import datasets and analyze them using Pandas functions.
 - To visualize data using different plots in Matplotlib.
-

Theory

NumPy

NumPy is a Python library used for numerical and scientific computations. It provides support for large multidimensional arrays and matrices along with various mathematical functions to operate on them efficiently.

Features of NumPy

- Fast array processing
 - Multidimensional array support
 - Mathematical and statistical functions
 - Memory efficient operations
-

Pandas

Pandas is a Python library used for data analysis and manipulation. It provides two major data structures:

- Series
- DataFrame

Features of Pandas

- Easy data handling
 - Data cleaning and filtering
 - CSV file handling
 - Statistical analysis tools
-

Matplotlib

Matplotlib is a visualization library in Python used to create graphs and plots.

Types of Graphs

- Line Plot
 - Scatter Plot
 - Bar Graph
 - Pie Chart
-

Software Requirements

- Python
- Jupyter Notebook / Google Colab
- NumPy
- Pandas
- Matplotlib

1. Creating 1-D and 2-D Arrays Using NumPy

```
In [1]: import numpy as np

arr1 = np.array([1, 2, 3, 4, 5])
print("1-D Array:")
print(arr1)

arr2 = arr1.reshape(1, 5)
print("Reshaped 2-D Array:")
print(arr2)

arr3 = np.array([[1, 2], [3, 4]])
arr4 = np.array([[5, 6], [7, 8]])

sum_array = arr3 + arr4

print("Addition of Arrays:")
print(sum_array)

print("Shape of arr3:")
print(arr3.shape)
```

```
1-D Array:
[1 2 3 4 5]
Reshaped 2-D Array:
[[1 2 3 4 5]]
Addition of Arrays:
[[ 6  8]
 [10 12]]
Shape of arr3:
(2, 2)
```

2. Accessing the Last Element of a 2-D Array

```
In [2]: print(arr3[-1, -1])
```

4

3. Array Slicing

```
In [3]: print(arr1[1:4])

print(arr3[:, 1])

print(arr1[::-1])
```

```
[2 3 4]
[2 4]
[5 4 3 2 1]
```

4. Searching Data Using where() and searchsorted()

```
In [4]: data = np.array([10, 20, 30, 40, 50])
```

```
x = np.where(data == 30)
print(x)

y = np.searchsorted(data, 35)
print(y)
```

```
(array([2]),)
3
```

5. Statistical Calculations

```
In [5]: from statistics import mode
import numpy as np

data = np.array([10, 20, 30, 40, 50])

mean = np.mean(data)
median = np.median(data)
mode_value = mode(data)
std = np.std(data)
variance = np.var(data)

print("Mean:", mean)
print("Median:", median)
print("Mode:", mode_value)
print("Standard Deviation:", std)
print("Variance:", variance)

actual = np.array([10, 20, 30])
predicted = np.array([12, 18, 33])

mse = np.mean((actual - predicted) ** 2)
rmse = np.sqrt(mse)

print("MSE:", mse)
print("RMSE:", rmse)
```

```
Mean: 30.0
Median: 30.0
Mode: 10
Standard Deviation: 14.142135623730951
Variance: 200.0
MSE: 5.666666666666667
RMSE: 2.3804761428476167
```

6. Creating DataFrame Using Dictionaries and NumPy Arrays

```
In [6]: import pandas as pd

data = {
    "Name": ["Ram", "Shyam", "Hari"],
    "Marks": [80, 75, 90]
}

df = pd.DataFrame(data)
```

```
print(df)
```

	Name	Marks
0	Ram	80
1	Shyam	75
2	Hari	90

7. Importing Dataset Using read_csv()

```
In [7]: # Import dataset from a CSV file
df = pd.read_csv('student_data.csv')

# Display the first few rows
print(df.head())

# Display basic information about the dataset
print(df.info())

# Display statistical summary
print(df.describe())
```

	ID	Name	Age	Marks
0	1	Aarav	20	85
1	2	Sita	21	90
2	3	Ram	22	88
3	4	Gita	20	76
4	5	Nabin	23	92

```
<class 'pandas.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 4 columns):
#   Column  Non-Null Count  Dtype
---  -
0    ID      5 non-null        int64
1   Name     5 non-null        str
2    Age     5 non-null        int64
3   Marks   5 non-null        int64
dtypes: int64(3), str(1)
memory usage: 292.0 bytes
None
```

	ID	Age	Marks
count	5.000000	5.000000	5.000000
mean	3.000000	21.200000	86.200000
std	1.581139	1.30384	6.26099
min	1.000000	20.000000	76.000000
25%	2.000000	20.000000	85.000000
50%	3.000000	21.000000	88.000000
75%	4.000000	22.000000	90.000000
max	5.000000	23.000000	92.000000

8. Adding New Columns in DataFrame

```
In [8]: df["Grade"] = ["A", "B", "A", "B", "A"]
print(df)
```

	ID	Name	Age	Marks	Grade
0	1	Aarav	20	85	A
1	2	Sita	21	90	B
2	3	Ram	22	88	A
3	4	Gita	20	76	B
4	5	Nabin	23	92	A

9. Data Visualization Using Matplotlib

```
In [9]: import matplotlib.pyplot as plt

subjects = [
    "Probability and Statistics",
    "Data Science and ML",
    "Artificial Intelligence",
    "Computer Graphics",
    "Engineering Economics",
    "Research Fundamentals"
]

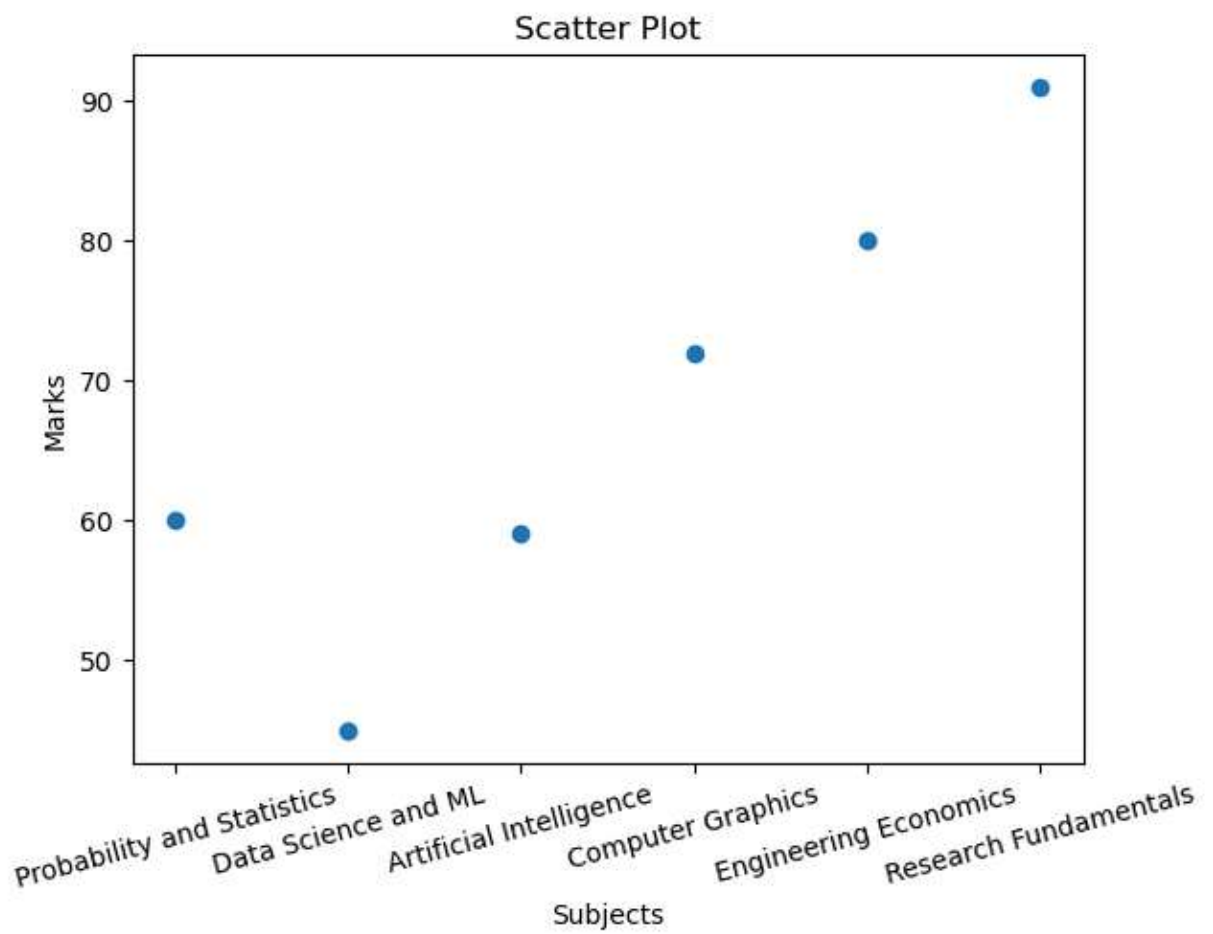
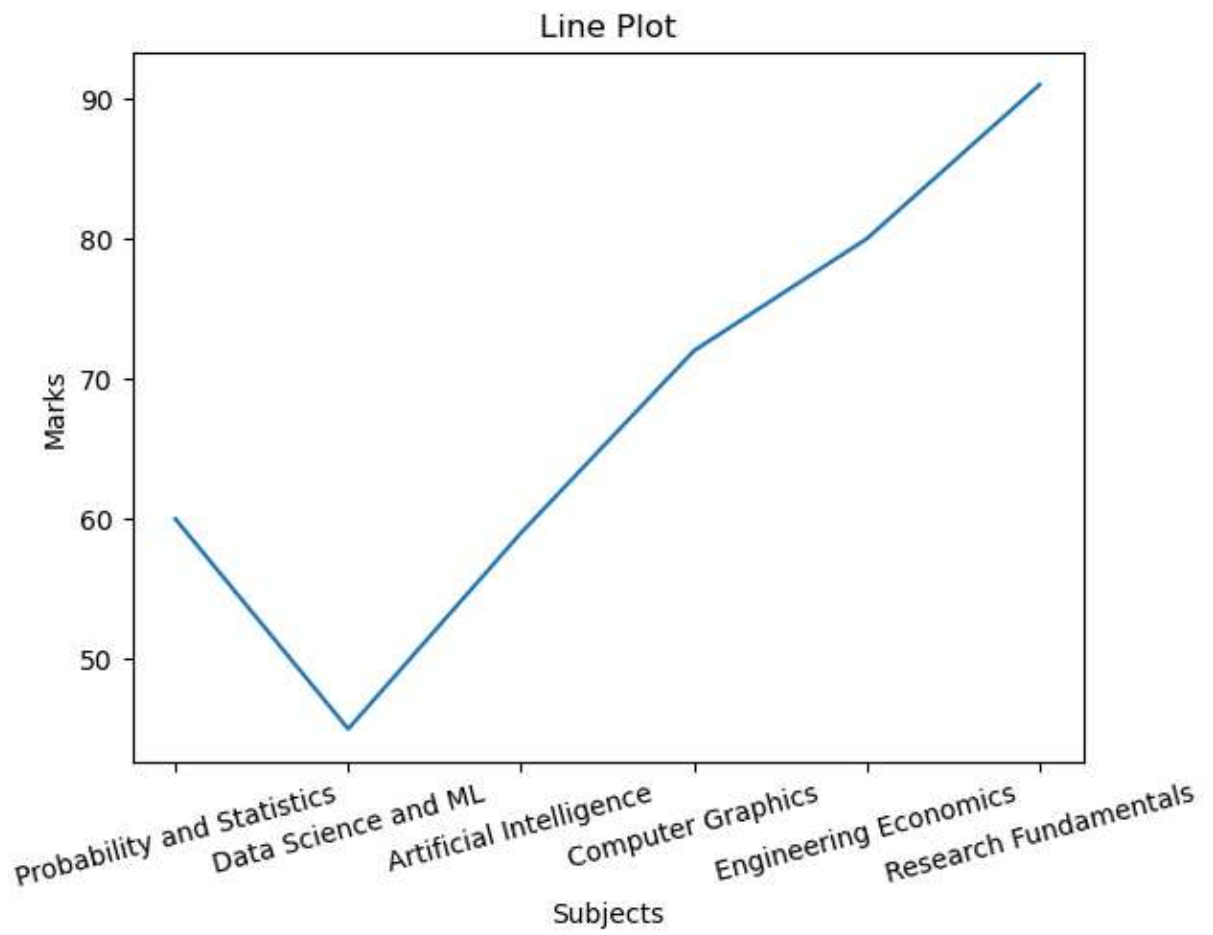
marks = [60, 45, 59, 72, 80, 91]

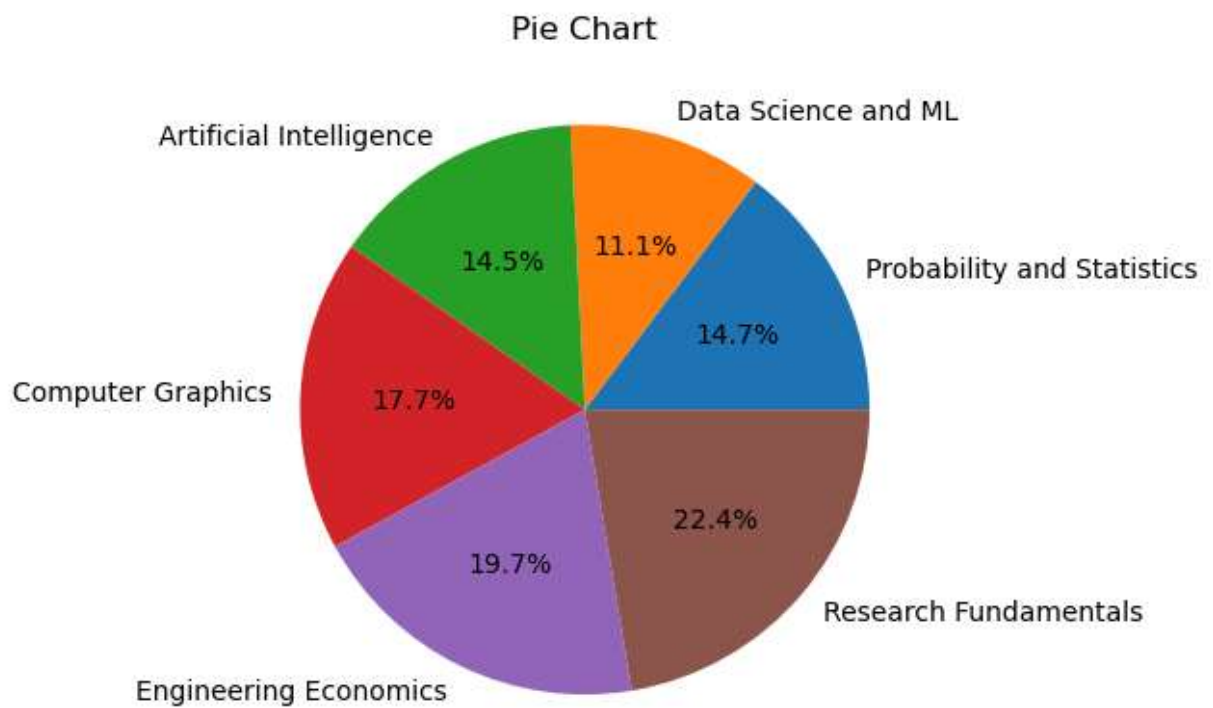
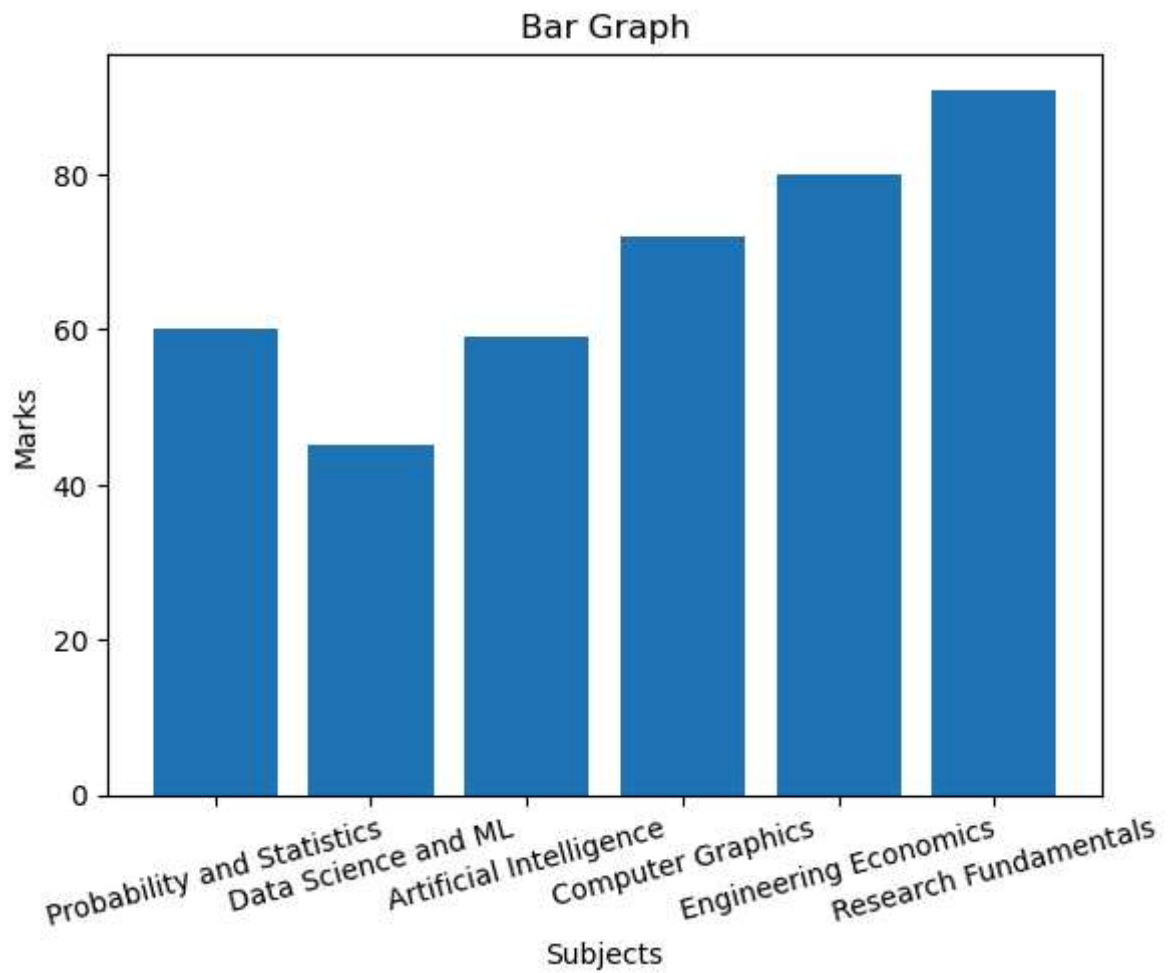
# Line Plot
plt.plot(subjects, marks)
plt.xlabel("Subjects")
plt.ylabel("Marks")
plt.title("Line Plot")
plt.xticks(rotation=15)
plt.show()

# Scatter Plot
plt.scatter(subjects, marks)
plt.xlabel("Subjects")
plt.ylabel("Marks")
plt.title("Scatter Plot")
plt.xticks(rotation=15)
plt.show()

# Bar Graph
plt.bar(subjects, marks)
plt.xlabel("Subjects")
plt.ylabel("Marks")
plt.title("Bar Graph")
plt.xticks(rotation=15)
plt.show()

# Pie Chart
plt.pie(marks, labels=subjects, autopct='%1.1f%%')
plt.title("Pie Chart")
plt.show()
```





Result

The lab was completed successfully. Arrays and DataFrames were created and manipulated using NumPy and Pandas. Statistical calculations were performed, and different plots were generated using Matplotlib.

Conclusion

This lab helped in understanding the fundamentals of NumPy, Pandas, and Matplotlib libraries. It provided practical knowledge of array manipulation, statistical analysis, data handling, and graphical visualization in Python, which are essential concepts in Data Science and Machine Learning.